
函数

复习

综合案例2

自助点餐系统

菜单

单点

套餐

结账

目录

1. 函数概述	了解
2. 函数类型	了解
3. 函数定义与声明	重点
4. 函数参数	重点
5. 引用	重点
6. 函数可变参数	熟悉
7. 函数默认参数	熟悉
8. 函数返回值	重点
9. 函数嵌套调用	熟悉
10. 函数递归	重点
11. 内联函数	熟悉
12. 内联函数与普通函数区别	了解
13. 函数重载	重点

函数概述

C++语言中的函数在其他编程语言中也称为过程或子例程。

我们可以创建函数来执行任何任务。

一个函数可以调用多次。

函数提供模块化和代码可重用性。

函数概述之优点

函数有很多优点，但这里主要介绍以下两点：

1. 提高代码可重用性

通过在C++中创建函数，可以调用函数多次。所以实现相同的功能不需要一遍又一遍地编写相同的代码。

2. 代码优化

函数能使代码优化，我们不需要写很多代码。假设，要检查多个数字(111,663和781等)是否是素数。如果不使用函数，需要编写计算质数逻辑程序3次。所以，这里就产生了不必要的重复代码。

函数类型

C++编程语言中有两种类型的函数：

1. **库函数**：是在C++头文件中声明的函数，如:ceil(x)，cos(x)，exp(x)等。
2. **用户定义的函数**：是由C++程序员创建的函数，以便可以多次使用它。它降低了大程序的复杂性并优化了代码。

函数定义与声明

定义语法

```
type funcName(paramlist)
{
    // todo
    return value;
}
```

声明语法

```
type funcName(paramlist);
```



有啥区别？

函数定义与声明

直接定义函数一般放在main函数之前，定义完即可使用

另外，可以在main函数前先声明，在main之后定义函数

函数定义与声明

```
void printName(){  
    cout << "c++" << endl;  
}  
  
int main(){  
  
    printName();  
    return 0;  
}
```

```
void printName();  
int main(){  
  
    printName();  
    return 0;  
}  
  
void printName(){  
    cout << "c++" << endl;  
}
```

函数4种基本形式

//1.无参数无返回值

```
void funcName()  
{  
    // 执行语句...  
}
```

//2.无参数有返回值

```
type funcName()  
{  
    // 执行语句...  
    return value;  
}
```

//3.有参数无返回值

```
void funcName(paramType1 param1)  
{  
    // 执行语句...  
}
```

//4.有参数有返回值

```
type funcName(paramType1 param1)  
{  
    // 执行语句...  
    return value;  
}
```

函数参数

语法

```
void funcName(paramType1 param1, paramType2 param2)
{
    // 执行语句...
}
```

定义了一个函数 funcName，该返回的返回值类型是 type，如果没有返回值，则写 void。

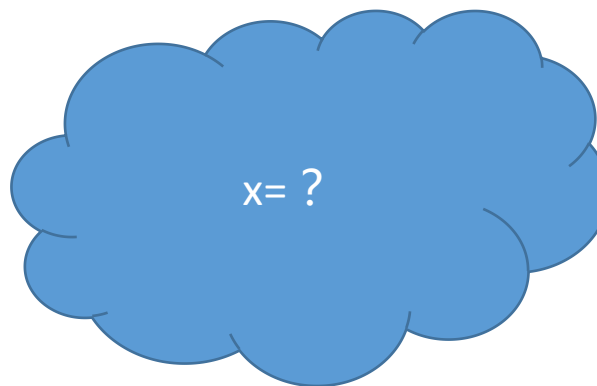
该函数有两个参数，分别为 paramType1 类型的参数 param1 和 paramType2 类型的参数 param2，函数的返回值为 void。

函数参数 案例

把任意一个整数，变成它的两倍。

函数参数 案例代码

```
#include <iostream>
using namespace std;
void change(int num)
{
    num *= 2;
}
int main()
{
    int x = 100;
    change(x);
    cout << "x= " << x << endl;
    return 0;
}
```

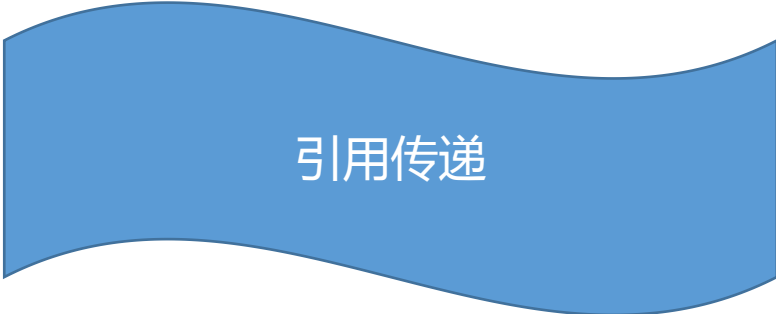


函数参数

定义了一个函数 `change()`，传入一个 `int` 类型的参数 `a`，在函数体中，对该函数的参数实现加 变成2倍 的操作。

最后，调用该函数，并传入变量 `x`，执行完 `change()` 函数之后，我们再次打印变量 `x` 的值，发现 `x` 的值还是 100，并不是乘2 之后的 200。

因为在 C++ 中，函数的参数是形式参数，即是实参的拷贝，所以修改的不是实参，所以 `x` 的值不会改变。



引用传递

引用概念

引用即为某个已存在的变量取别名，引用变量与被引用变量公用一块内存空间，比如土豆和马铃薯都是同一种东西的不同命名。

通过在数据类型后、变量名前添加 “&” 符号来定义引用类型。

语法

```
type &name = data ;
```

```
int num = 11;
```

```
int &num2 = num; //num2 就是num的别名,num2就是num的引用
```

引用特性

- 1 引用只是别名，不占内存；和它引用的变量共用同一块内存空间；
- 2 引用仅在定义时带&,使用时普通变量一样使用,不含&;
- 3 引用必须在创建的时候初始化，一旦引用初始化后，就不能改变引用所指向的变量，类似于指针常量。
- 4 引用必须与一个确定的合法内存单元相关联,不存在NULL引用且不可以使用；

一个变量可以有多个引用，就像人一样，有可能有多个昵称！

引用特性

```
int num1 = 100 ;
```

```
int &num2 = num1;
```

```
int num3 = 200 ;
```

```
num2 = num3;
```

```
num1 = ? num2 = ? num3 = ?
```

引用使用场景

1 做参数

```
void change(int & v1,int & v2){}
```

在函数被调用时，就相当于传递实参

引用传递

```
#include <iostream>

using namespace std;

void change(int & num)
{
    num *= 2;
}

int main()
{
    int x = 100;
    change(x);
    cout << "x= " << x << endl;
    return 0;
}
```

引用作为返回值

语法

```
type& funcName(){}  

```

当函数返回一个引用时，则返回一个指向返回值的隐式**指针**

当返回一个引用时，要注意被引用的对象**不能超出作用域**。所以返回一个对局部变量的引用是不合法的，但是，可以返回一个对静态变量的引用。

引用作为返回值

```
int& func(int& v1) {  
    v1 *=100;  
    return v1;  
}  
  
int main() {  
    int num = 2;  
    cout << func(num) << endl;  
    cout << num << endl;  
    return 0;  
}
```

形参与实参

形参：函数括号里面的变量参数，叫形式参数

```
type funcName(type v1,type v2);
```

实参：调用函数时给的实值，叫实际参数

```
funcName(2,3);
```

函数可变参数

语法

```
type funcName(paramType1 param1, ...)  
{  
    // 执行语句...  
    return value ;  
}
```

函数可变参数

可变参数，即函数参数的个数是任意的，最典型的可变参数的就是系统内置的 `scanf` 函数和 `printf` 函数。

可变参数的函数必须至少有一个强制参数，可选参数的类型可以变化。可选参数的数量由强制参数的值决定，或由用来定义可选参数列表的特殊值决定。

对于每一个强制参数来说，函数头部都会显示一个适当的参数，像普通函数声明一样。参数列表的格式是强制性参数在前，后面跟着一个逗号和省略号（`...`），这个省略号代表可选参数。

函数可变参数 使用

可变参数函数要获取可选参数时，必须通过一个类型为 `va_list` 的对象，它包含了参数信息。这种类型的对象也称为参数指针（argument pointer），它包含了栈中至少一个参数的位置。

可以使用这个参数指针从一个可选参数移动到下一个可选参数，由此，函数就可以获取所有的可选参数。

`va_list` 类型被定义在头文件 `stdarg.h` 中。

当编写支持参数数量可变的函数时，必须用 `va_list` 类型定义参数指针，以获取可选参数。当我们处理可变参数时，主要涉及到以下几个宏函数：

`va_start`

语法

```
void va_start(va_list argptr, lastparam);
```

`argptr` 定义好的 `va_list` 变量。

`lastparam` 强制参数。

宏 `va_start` 使用第一个可选参数的位置来初始化 `argptr` 参数指针。该宏的第二个参数必须是该函数最后一个有名称参数的名称。必须先调用该宏，才可以开始使用可选参数

函数可变参数 使用

va_arg

语法

```
type va_arg(va_list argptr, type);
```

argptr 定义好的 va_list 变量。

type 可变参数的具体类型。

展开宏 va_arg 会得到当前 argptr 所引用的可选参数，也会将 argptr 移动到列表中的下一个参数。宏 va_arg 的第二个参数是刚刚被读入的参数的类型。

函数可变参数 使用

va_end

语法

```
void va_end(va_list argptr);
```

argptr 定义好的 va_list 变量。

当不再需要使用参数指针时，必须调用宏 va_end。如果想使用宏 va_start 或者宏 va_copy 来重新初始化一个之前用过的参数指针，也必须先调用宏 va_end。

函数可变参数 使用

va_copy

语法

```
void va_copy(va_list dest, va_list src);
```

dest 目的 va_list 变量。

src 源 va_list 变量。

宏 va_copy 使用当前的 src 值来初始化参数指针 dest。然后就可以使用 dest 中的备份获取可选参数列表，从 src 所引用的位置开始。

函数可变参数 案例

用函数可变参数，实现求任意变量的和。

函数可变参数 代码

```
#include <cstdio>
#include <stdarg.h>
#include <iostream>
using namespace std;
int mulSum(int n, ...)
{
    int sum = 0;
    va_list argptr;
    va_start(argptr, n);
    for (int i = 0; i < n; i++)
    {
        sum += va_arg(argptr, int); //累加到 sum 中
    }
    va_end(argptr);
    return sum;
}
int main()
{
    int result = mulSum(4, 1, 3, 5, 8);
    cout << "result = " << result << endl;
    return 0;
}
```

// 初始化argptr
// 对每个可选参数，读取类型为int

函数默认参数

定义函数时可以给形参指定一个默认的值，这样调用函数时如果没有给这个形参赋值（没有对应的实参），那么就使用这个默认的值。也就是说，调用函数时可以省略有默认值的参数。如果用户指定了参数的值，那么就使用用户指定的值，否则使用参数的默认值。

所谓默认参数，指的是当函数调用中省略了实参时自动使用的一个值，这个值就是给形参指定的默认值。

函数默认参数

语法

```
type funcname(paramterType param1, paramterType2 param2 = value)
{
    return [val];
}
```


默认参数案例

```
#include <iostream>
using namespace std;
int sum(int num1, int num2 = 100)
{
    int ret = num1 + num2;
    cout << ret << endl;
}
int main()
{
    int num1 = 1;
    int num2 = 2;
    sum(num1, num2);
    sum(num1);
    return 0;
}
```

函数返回值

C++ 中函数 **可以不返回任何值**，也可以返回一个值，但 C++ 的函数 **不支持返回多个值**。同时，C++ 函数的返回值需要显式的声明其类型。

语法

```
type funcName(paramType1 param1, paramType2 param2)
{
    // 执行语句...
    return value;
}
```

函数返回值 案例

输入两个整数v1,v2,交换v1和v2的值。

函数返回值 代码

```
int swap(int num1,int num2){  
    int num3= num1;  
    num1 = num2;  
    num2 = num1;  
    return num1,num2;   
}
```

```
void swap(int num1,int num2){  
    int num3= num1;  
    num1 = num2;  
    num2 = num1;  
    cout <<num1 << endl;  
    cout <<num2 << endl;  
}
```

函数嵌套调用

函数可以嵌套调用，即，我们可以在一个函数里面调用另一个函数。

语法

```
void funcName()  
{  
    funcName1();  
    funcName2();  
    [return]  
}
```

函数嵌套调用 案例

函数嵌套调用，实现求阶乘的和

函数嵌套调用 代码

```
#include <iostream>
using namespace std;
int factorial(int num)
{
    int ret = 1;
    for(int i = 1; i <= num; i++)
    {
        ret *= i;
    }
    return ret;
}
int factorialSum(int n)
{
    int sum = 0;
    for(int i = 1; i <= n; i++)
    {
        sum += factorial(i);
    }
    return sum;
}
int main()
{
    int factorialSumRet = factorialSum(10);
    cout << "factorialSumRet = " << factorialSumRet << endl;
    return 0;
}
```

函数递归

函数递归就是一个 函数 在函数体内又调用了自身，称为函数的递归调用。

函数递归

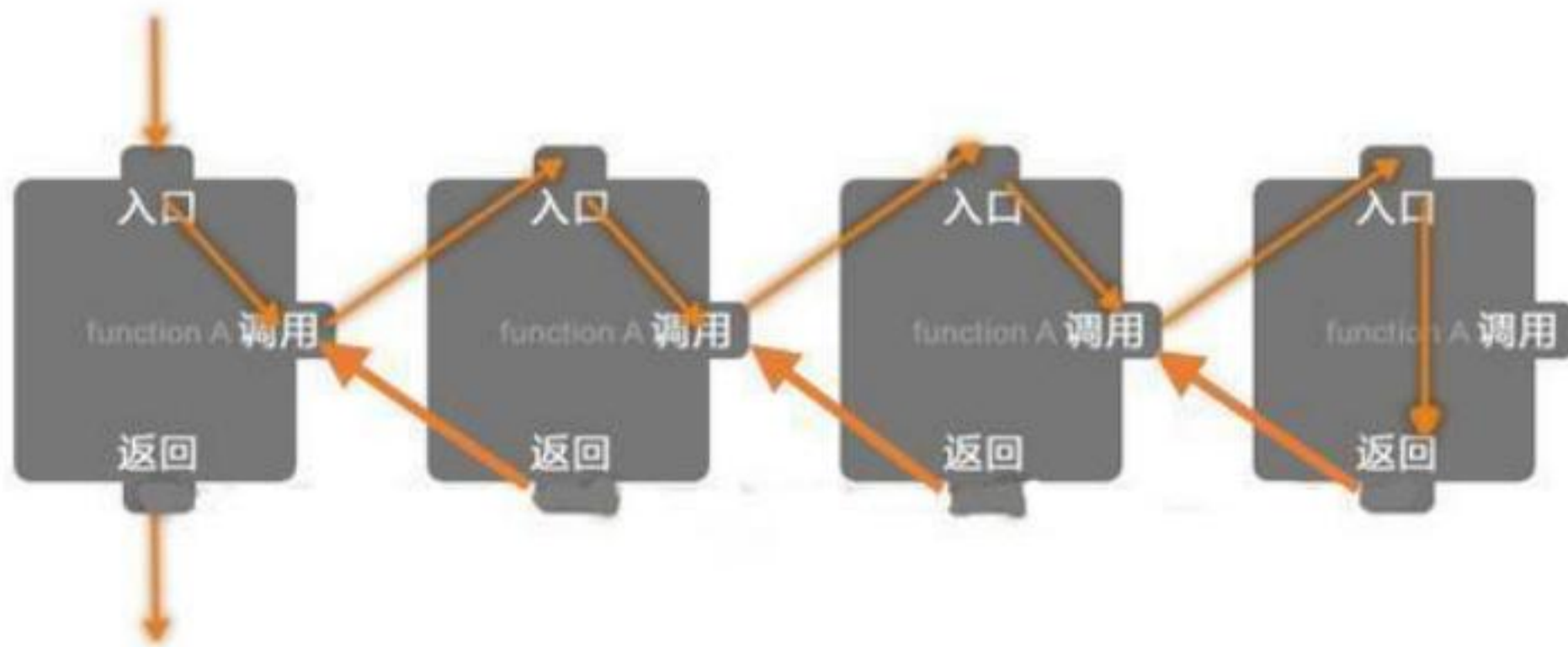
语法

```
type funcName(param)
{
    if (param == cond)
    {
        return;
    }
    funcName(param2);
}
```

递归条件

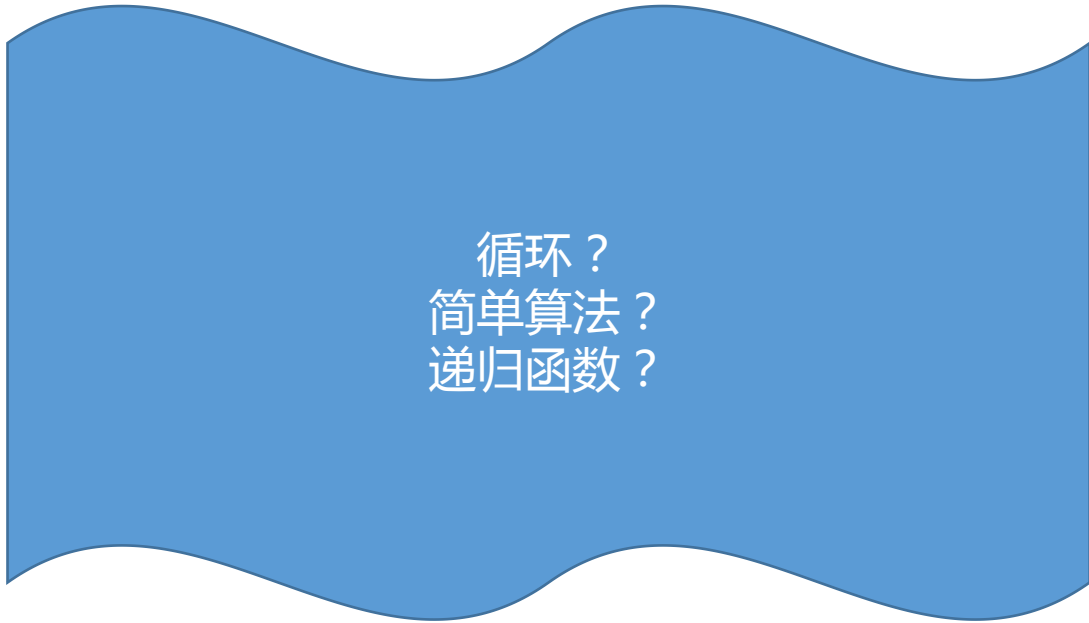
1. 执行一个函数时，就创建一个新的受保护的独立空间(新函数栈)。
2. 函数的局部变量是独立的，不会相互影响。
3. 递归必须向退出递归的条件逼近，否则就是无限递归了。
4. 当一个函数执行完毕，或者遇到 return，就会返回，遵守谁调用，就将结果返回给谁，同时当函数执行完毕或者返回时，该函数本身也会被系统销毁。

递归图示



递归案例

求 $1+2+3+4+5+\dots+n$ 的和



循环？
简单算法？
递归函数？

循环实现代码

```
int sum(int n){  
    int total = 0;  
    for(int i=1; i<=n;i++){  
        total += i;  
    }  
    return total;  
}
```

简单算法实现代码

```
int sum(int n){  
    return ( 1+n ) *n /2.0;  
}
```

递归实现代码

```
int sum(int n){  
    if(n==1){  
        return 1;  
    }  
    return sum(n-1) + n;  
}
```

递归案例2

递归实现斐波那契数列（ fibo ）的第n项。

递归案例2 分析

1 1 2 3 5 8 13 21 34 55 89 ...

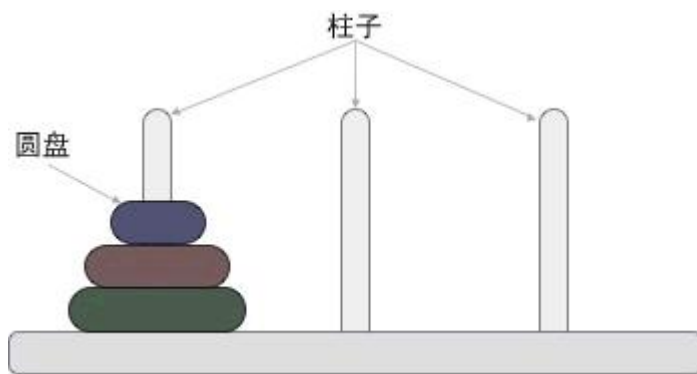
递归案例2 代码

```
int fibo(int n){  
    if(n <= 2){  
        return 1;  
    }  
    else{  
        return fibo(n-1) + fibo(n-2);  
    }  
}
```

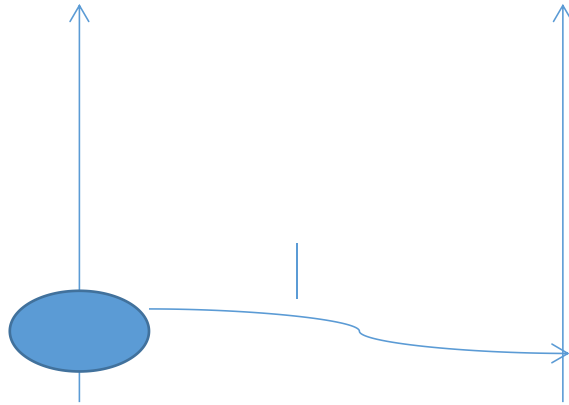
递归案例3 汉罗塔

移动过程中必须遵守以下规则：

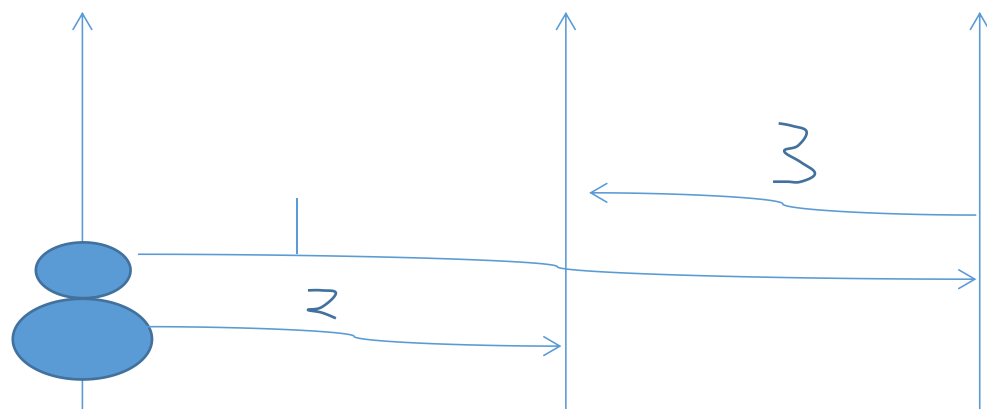
- 1.每次只能移动柱子最顶端的一个圆盘；
- 2.每个柱子上，小圆盘永远要位于大圆盘之上.



递归案例3 汉罗塔 分析



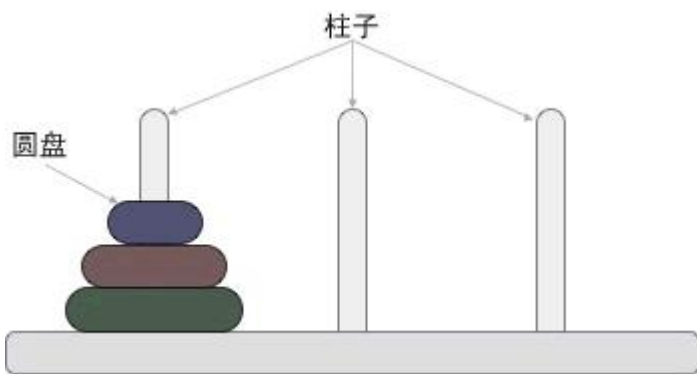
递归案例3 汉罗塔 分析



递归案例3 汉罗塔 分析

对于 n 个圆盘的汉诺塔问题，移动圆盘的过程是：

- 1 将起始柱上的 $n-1$ 个圆盘移动到辅助柱上；
- 2 将起始柱上遗留的 1 个圆盘移动到目标柱上；
- 3 将辅助柱上的所有圆盘移动到目标柱上。



递归案例3 汉罗塔 代码

```
void hanoi(int num, char sou, char tar, char aux) {  
    static int count = 1; //统计移动次数  
    //如果圆盘数量仅有 1 个，则直接从起始柱移动到目标柱  
    if (num == 1) {  
        cout << "第" << count << "次:从" << sou << " 移动至" << tar << endl;  
        count++;  
    }  
    else {  
        //递归调用 hanoi() 函数，将 num-1 个圆盘从起始柱移动到辅助柱上  
        hanoi(num - 1, sou, aux, tar);  
        //将起始柱上剩余的最后一个大圆盘移动到目标柱上  
        cout << "第" << count << "次:从" << sou << " 移动至" << tar << endl;  
        count++;  
        //递归调用 hanoi() 函数，将辅助柱上的 num-1 圆盘移动到目标柱上  
        hanoi(num - 1, aux, tar, sou);  
    }  
}  
  
int main()  
{  
    //以移动 3 个圆盘为例，起始柱、目标柱、辅助柱分别用 A、B、C 表示  
    hanoi(3, 'A', 'B', 'C');  
    return 0;  
}
```

内联函数

在 C 语言中，如果一些函数被频繁调用，不断地有函数入栈，即函数栈，会造成栈空间或栈内存的大量消耗。在 C++ 中，为了解决这个问题，特别的引入了 `inline` 修饰符，表示为内联函数。

栈空间就是指放置程式的局部数据也就是函数内数据的内存空间，在系统下，**栈空间是有限的**，假如频繁大量的使用就会造成因栈空间不足所造成的程式出错的问题，函数的**死循环 递归调用**的最终结果就是导致栈内存空间枯竭。

内联函数使用场景

函数是一个可以重复使用的代码块，CPU 会一条一条地挨着执行其中的代码。CPU 在执行主调函数代码时如果遇到了被调函数，主调函数就会暂停，CPU 转而执行被调函数的代码；被调函数执行完毕后再返回到主调函数，主调函数根据刚才的状态继续往下执行。

一个 C/C++ 程序的执行过程可以认为是多个函数之间的相互调用过程，它们形成了一个或简单或复杂的调用链条，这个链条的起点是 `main()`，终点也是 `main()`。当 `main()` 调用完了所有的函数，它会返回一个值（例如 `return 0;`）来结束自己的生命，从而结束整个程序。

函数调用是有时间和空间开销的。程序在执行一个函数之前需要做一些准备工作，要将实参、局部变量、返回地址以及若干寄存器都压入栈中，然后才能执行函数体中的代码；函数体中的代码执行完毕后再还要清理现场，将之前压入栈中的数据都出栈，才能接着执行函数调用位置以后的代码。

如果函数体代码比较多，需要较长的执行时间，那么函数调用机制占用的时间可以忽略；如果函数只有一两条语句，那么大部分的时间都会花费在函数调用机制上，这种时间开销就就不容忽视。

为了消除函数调用的时空开销，C++ 提供一种提高效率的方法，即在编译时将函数调用处用函数体替换，类似于 C 语言中的宏展开。这种在函数调用处直接嵌入函数体的函数称为内联函数（Inline Function），又称内嵌函数或者内置函数。

内联函数定义

定义

关键字 `inline` 必须与函数定义体放在一起才能使函数成为内联，仅将 `inline` 放在函数声明前面不起任何作用。

语法

```
inline returntype funcname(paramterType params) // inline 与函数定义体放在一起
{
    return [val];
}
```

技术细节

如下的函数风格，不是内联函数，因为内联函数的 inline 关键字，只能放在函数定义的地方：

```
inline void Foo(int x, int y); // inline 仅与函数声明放在一起  
void Foo(int x, int y)  
{  
}
```

inline关键字

inline 是一种 “用于实现的关键字”，而不是一种 “用于声明的关键字”。一般地，用户可以阅读函数的声明，但是看不到函数的定义。尽管在大多数教科书中内联函数的声明、定义体前面都加了 inline 关键字，但我认为 inline 不应该出现在函数的声明中。

这个细节虽然不会影响函数的功能，但是体现了高质量 C++/C 程序设计风格的一个基本原则：声明与定义不可混为一谈，用户没有必要、也不应该知道函数是否需要内联

使用限制

inline 的使用是有所限制的，inline 只适合函数体内代码简单的函数使用，不能包含复杂的结构控制语句例如 while、switch 并且内联函数本身不能是直接递归函数(自己内部还调用自己的函数)

内联函数 案例

判断多个数是否为偶数？

内联函数 代码

```
#include <iostream>
using namespace std;
inline bool isEven(int num)
{
    return num % 2 == 0;
}
int main()
{
    int num1 = 100;
    int num2 = 101;
    bool b1 = isEven(num1);
    bool b2 = isEven(num2);
    cout << "b1 = " << b1 << endl;
    cout << "b2 = " << b2 << endl;
    return 0;
}
```

内联函数与普通函数区别

内联函数 和 **普通函数** 最大的区别在于内部的实现方面，当普通函数在被调用时，系统首先跳跃到该函数的入口地址，执行函数体，执行完成后，再返回到函数调用的地方，函数始终**只有一个拷贝**。

而内联函数则不需要进行一个寻址的过程，当执行到内联函数时，此函数展开（很类似宏的使用），如果在 N 处调用了此内联函数，则此函数就会有**N 个代码段的拷贝**。

函数重载

在 C 中，我们定义的 变量名 和函数名在同一作用域一定是不允许重复的，因此，如果我们需要定义几个功能类似的函数，那么我们必须想办法给它们做不同的命名。

在 C++ 中，允许我们为函数设置同样的名字，但这些函数的参数列表的顺序或者类型必须不一致，这就是 C++ 中的函数重载。

函数重载

函数重载是 C++ 允许在同一作用域中声明几个类似的同名函数，这些同名函数的形参列表（参数个数，类型，顺序）必须不同，常用来处理实现功能类似数据类型不同的问题。在 C++ 中不仅函数可以重载，运算符也可以重载。

函数重载规则

1. 函数名称必须相同。
2. 参数列表必须不同（**个数不同、类型不同、参数排列顺序不同等**）。
3. 函数的返回类型可以相同也可以不相同。
4. 仅仅返回类型不同不足以成为函数的重载。

如何做到函数重载

C++ 代码在编译时会根据参数列表对函数进行重命名，例如 `void Swap(int a, int b)` 会被重命名为 `_Swap_int_int`，`void Swap(float x, float y)` 会被重命名为 `_Swap_float_float`。

当发生函数调用时，编译器会根据传入的实参去逐个匹配，以选择对应的函数，如果匹配失败，编译器就会报错，这叫做重载决议（Overload Resolution）。从这个角度讲，函数重载仅仅是语法层面的，本质上它们还是不同的函数，占用不同的内存，入口地址也不一样。

函数重载案例

利用函数重载，实现求两个数的和

函数重载 代码

```
int sum(int a,int b){
```

```
    return a+b;
```

```
}
```

```
float sum(float a,float b){
```

```
    return a+b;
```

```
}
```

```
double sum(double a,double b){
```

```
    return a+b;
```

```
}
```

课堂练习

利用函数求出num1-num2之间的所有素数 (num1 < num2).

课堂练习

利用函数求出num1-num2之间的所有素数 (num1 < num2).



素数是一个大于1，并且只能被1和本身整除

课堂练习代码

```
bool isPrime ( int n){ // 判断是否为素数
    bool flag = 1;
    for(int i=2; i<=n/2;i++){
        if(n % i == 0){
            return 0;
        }
    }
    return 1;
}
```

课堂练习代码

```
void prime ( int num1,int num2){ // 输出num1 -num2之间的素数
    for(int i = num1; i<=num2; i++){
        if(isPrime(i)){
            cout << i << " ";
        }
    }
}
```


课堂练习代码

```
int main(){  
    int num1 ,num2;  
    cin >> num1 >> num2;  
    prime(num1,num2);  
    return 0;  
}
```

知识点总结

1.	函数概述	了解
2.	函数类型	了解
3.	函数定义与声明	重点
4.	函数参数	重点
5.	引用	重点
6.	函数可变参数	熟悉
7.	函数默认参数	熟悉
8.	函数返回值	重点
9.	函数嵌套调用	熟悉
10.	函数递归	重点
11.	内联函数	熟悉
12.	内联函数与普通函数区别	了解
13.	函数重载	重点

作业

求阶乘的和

【题目描述】

给定正整数 n ，求不大于 n 的正整数的阶乘的和（即求 $1!+2!+3!+\dots+n!$ ），输出阶乘的和。

【输入】

输入有一行，包含一个正整数 n （ $1 < n < 12$ ）。

【输出】

输出有一行：阶乘的和。

【输入样例】

5

【输出样例】

153



用递归函数

下节课预告

变量作用域和生存周期